

Chapter 01 — M1: Folder Structure Analysis

Feature type: Mandatory analysis module (M1). **Source:** `backend/src/ai/nodes/m1Folder.node.ts` ,
`backend/src/ai/prompts/m1Prompt.ts` **Output type:** `IFolderEntry[]`
(`backend/src/models/repoAnalysis.model.ts`)

1. What it does

M1 looks at the repository's directory layout and, for **each unique folder**, produces:

- `path` — the directory path (e.g. `src/controllers`)
- `purpose` — a concise 1-sentence description of what that folder is for
- `type` — one of `entry` | `logic` | `config` | `utility` | `test` | `other`

This is the "what am I looking at" layer: a developer opening an unknown repo gets a plain-English map of the folder tree.

2. How it works (step by step)

```
parsedRepo.sourcePaths          e.g. ["src/controllers/x.ts", "src/config/db.ts", ...]
  |
  ▼
derive unique folder paths      p.split("/").slice(0,-1).join("/") → dedupe (Set)
  |
  ▼
guard: no folders? → return [] + error "[M1] No folders found in parsed repo"
  |
  ▼
LLM call (M1Model.withStructuredOutput(M1OutputSchema))
  system: M1_System_Prompt      (category rules, 1-sentence constraint)
  user:   buildM1UserPrompt(folderPaths, sourcePaths)
  |
  ▼
result.folders → m1Result : IFolderEntry[]
```

Token-saving trick: the node feeds the model only the **unique directory paths** (not every file), so folder-level reasoning isn't drowned in file-level noise.

3. The model behind it

- **Model:** `M1Model = NVIDIA nemotron-3-nano-30b-a3b`, `temperature: 0`.
- **Why the nano model:** M1 is simple classification — the smallest/fastest Nemotron is enough. (M2/M3 use the larger `super-120b`.)
- Structured output is enforced by `withStructuredOutput(M1OutputSchema)` (Zod), so the model must return the `{ folders: [...] }` shape or LangChain rejects it.

4. Categories (the `type enum`)

Category	Meaning (as prompted)
<code>entry</code>	Application entry / bootstrap directory
<code>logic</code>	Core business logic (controllers, services, domain)
<code>config</code>	Configuration, env, build setup
<code>utility</code>	Helpers, shared utils
<code>test</code>	Tests / specs
<code>other</code>	Anything that doesn't fit above (schema default)

5. Failure behavior

- Empty folder list → returns `m1Result: []` + a non-fatal error string; the graph continues.
 - Any thrown error → caught, returns `m1Result: []` + `[M1] <message>`; **does not** crash M2/M3.
 - If M1 is missing at the Combine step, the final result carries `m1: []` and a `[Combine] M1 result missing` warning.
-

6. Where it surfaces in the UI

`data.m1[]` → frontend `folderHierarchy` (folder tree in **RepositoryStructurePage** and the Dashboard). `type === "entry"` marks a folder as starred/highlighted.

7. Known gaps / improvement ideas

- Purpose text quality depends entirely on folder names — no file-content signal feeds M1.
- No dedup of near-identical purposes across sibling folders.
- Categories are single-label; a folder that is both `logic` and `entry` must pick one.